



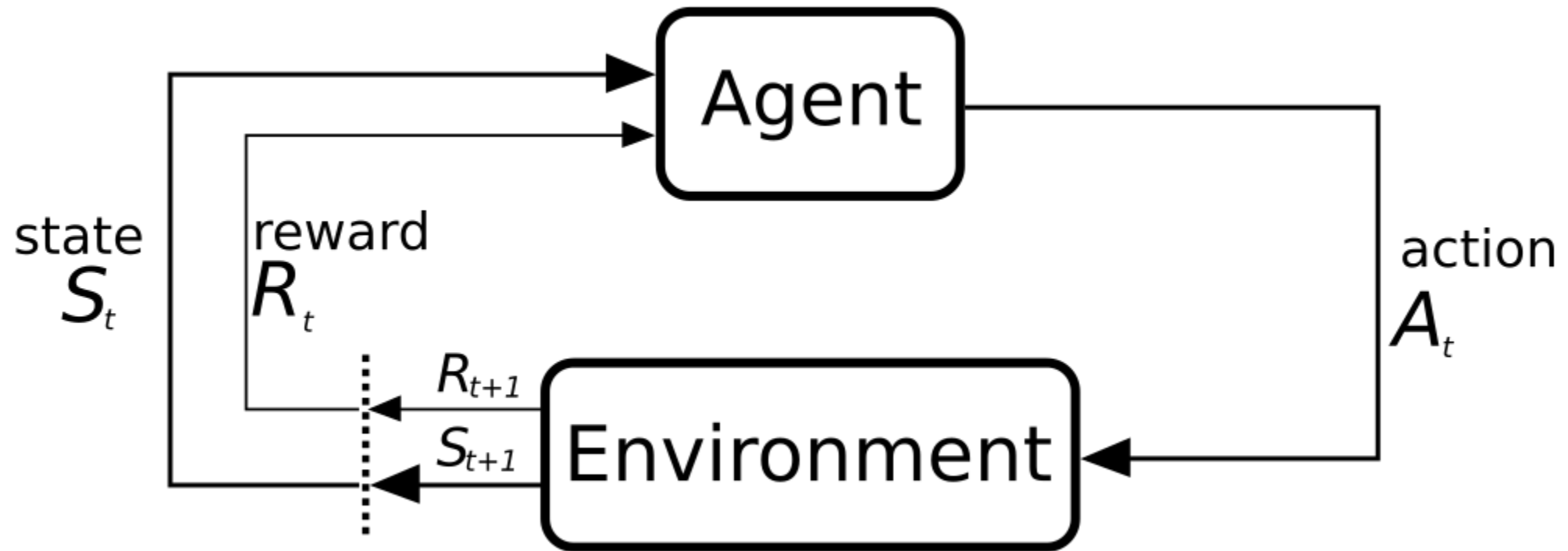
Exploration & Exploitation

Blink Sakulkueakulsuk

Recap

- A way to learn – a natural one
 - Learning through interactions
 - Active learning
 - Sequential interaction
 - Goal-directed
 - Learn even without the optimal sequence of actions

Recap



Rewards

It means, at time t , how good is the agent action

The ultimate goal of the agent is to maximize cumulative reward

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

This cumulative reward is called **return**

Values

We define the expected cumulative reward of a state s , the **value**.

$$\begin{aligned} v(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s] \end{aligned}$$

Action values

We can map value to both action and state

$$\begin{aligned} q(s, a) &= \mathbb{E}[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s, A_t = a] \end{aligned}$$

Environment State

The internal state of the environment

Usually, it contains a lot of information. Those information may or may not be useful for the agent.

Sometimes, it is not even visible to the agent.

Agent State

A state of the agent can be reached by its **history**

$$H_t = O_0, A_0, R_1, O_1, \dots, O_{t-1}, A_{t-1}, R_t, O_t$$

* O = Observation

Once the agent reaches this state, the actions can be determined

Markov decision processes

$$p(r, s \mid S_t, A_t) = p(r, s \mid H_t, A_t)$$

The equation above essentially means “the future state is independent of the past state given the present state”

MDP provides baseline for reinforcement learning.

Agent State

A state of the agent in this case is **a function of history**

$$S_{t+1} = u(S_t, A_t, R_{t+1}, O_{t+1})$$

The agent state is typically much smaller than the environment state

Policy

Given a state, a policy $\pi(S)$ defines an agent's action

- Mapping from agent state to action

Deterministic policy: $\pi(S) = A$

Stochastic policy: $\pi(A|S) = p(A|S)$

Value function

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}[G_t \mid S_t = s, A_t \sim \pi(s)] \\&= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t \sim \pi(s)]\end{aligned}$$

- $A_t \sim \pi(s)$ means that we choose action A from policy π
- $\gamma \in [0,1]$ is a discount factor

This is called a **Bellman equation**

If we want to find the optimal value, we can modify the equation to

$$v_*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

Value function approximations

In many cases, we cannot find, or it is not feasible to find, the exact value from the value function.

- The states are infinitely many.

We need to approximate the function.

Model

A **model** predicts what will happen next in the environment

For example, a model P predicts the next state

$$P(s, a, s') \approx p(S_{t+1} = s' \mid S_t = s, A_t = a)$$

For example, a model R predicts the next reward

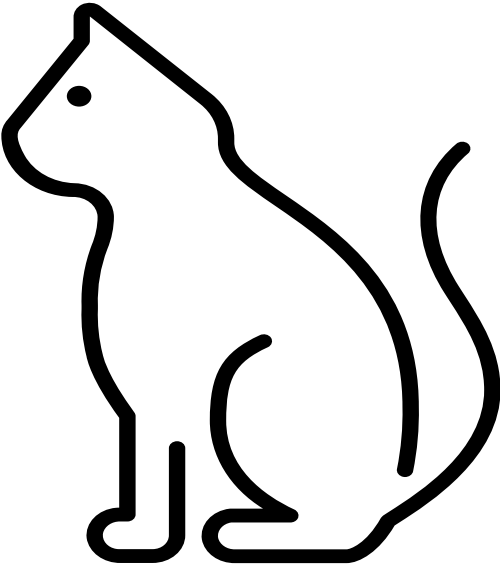
$$R(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

Exploration & Exploitation

Key Question

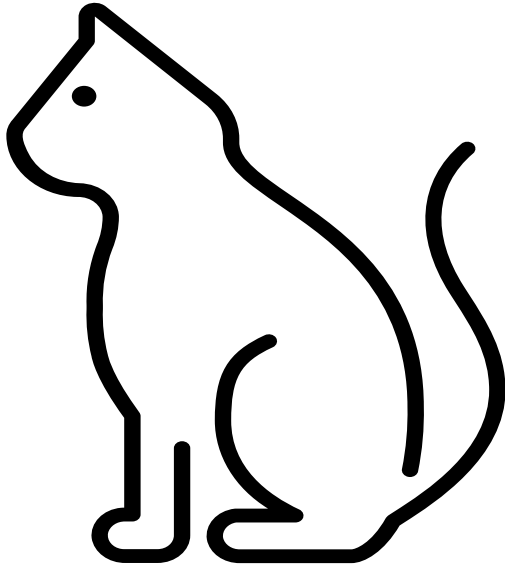
How can we **learn** without compromising the **reward**?

Example

Day	Action	Reward	
1			
2			
3	?		

Example

Day	Action	Reward
1		
2		
3		
4	?	



Exploration vs Exploitation

Exploration: gain knowledge

Exploitation: gain maximum reward

But do we have enough knowledge to know the action to gain maximum reward?

- It is a trade-off. We need to learn, but not too much.

Introducing Multi-Armed Bandit

AKA Slot Machine

At each step t the agent performs an action $A_t \in \mathcal{A}$

- The agent receives a reward R_t
- The agent doesn't know the reward rate for each action.
(each machine has its own reward distribution $p(r|a)$)
- The agent wants the maximum return over the lifetime.



Action values

Since we have only 1 state, the action value is the following equation.

$$q(a) = \mathbb{E}[R_t | A_t = a]$$

$$Q_t(a) = \frac{\text{sum of reward}}{\text{numbers of action}}$$

Expectation can be found as the average reward of the action.

$$Q_t(a) = \frac{\sum_{n=1}^t R_n I(A_n = a)}{\sum_{n=1}^t I(A_n = a)}$$

Given $I(True) = 1$ and $I(False) = 0$

Action values

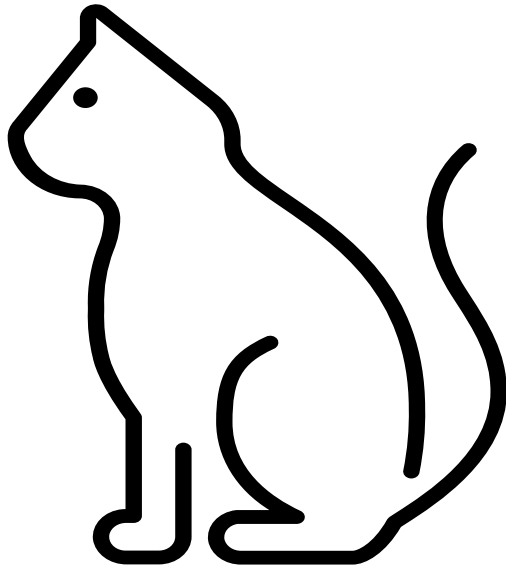
Or it can be updated iteratively:

$$Q_t(A_t) = \begin{cases} Q_{t-1}(A_t) + \alpha_t \overset{\text{error}}{(R_t - Q_{t-1}(A_t))} & \text{for } A_t \\ Q_{t-1}(a) & \text{for } a \neq A_t \end{cases}$$

Given

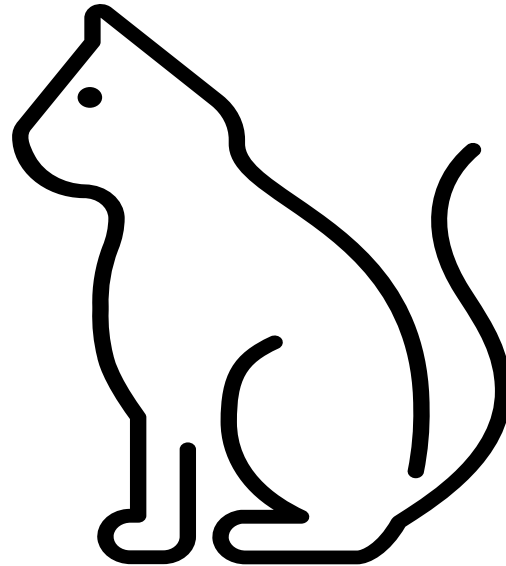
$$\alpha_t = \frac{1}{N_t(A_t)}, \quad N_t(A_t) = N_{t-1}(A_t) + 1, \quad N_t(a) = 0, \forall a$$

Example

Day	Action	Reward		Food: $R = 1$ No food: $R = -1$
1				
2				$Q_3(\text{white}) = 0$ $Q_3(\text{black}) = -1$
3				
4	?			

Example

Day	Action	Reward
1		
2		
3		
4		
5		
6		



Food: $R = 1$

No food: $R = -1$

$Q_6(\text{white}) = -0.6$

$Q_6(\text{black}) = -1$

What choice to choose next?

Introducing Regrets

At the point, everyone knows that we need to balance between exploration and exploitation.

- How can we balance it?
- Can we set some assumptions and follow them?
- Can we even do it optimally?

Regrets

The optimal value

$$v_* = \max_{a \in A} q(a) = \max_a \mathbb{E}[R_t | A_t = a]$$

Regret is the loss occurred when taking sub-optimal action

$$v_* - q(A_t)$$

Noted that the agent doesn't know the regret, because it doesn't know the optimal value

Regrets

We can sum up all the regrets in the agent's lifetime

$$L_t = \sum_{i=1}^t (v_* - q(a_i))$$

By minimizing the cumulative regret, we maximize the cumulative reward

Regret Example

Day	Action	Reward
-----	--------	--------

1



2



3



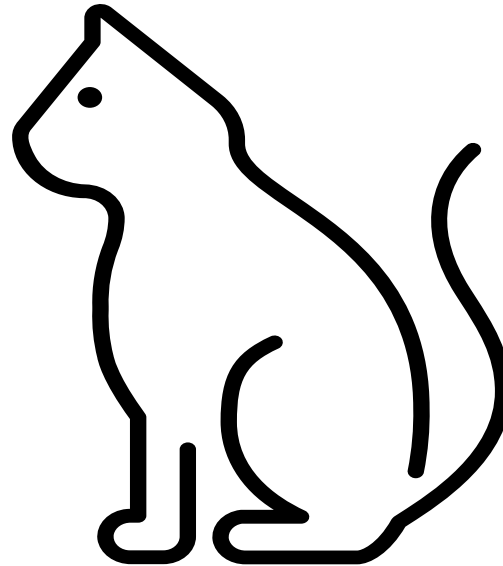
4



5



6



Food: $R = 1$ / No food: $R = -1$

$$p(\text{food}|\text{white}) = 0.1$$

$$p(\text{food}|\text{black}) = 0.9$$

$$v_* = q(\text{black}) = 0.9 * 1 + 0.1 * (-1) = 0.8$$

$$q(\text{white}) = 0.9 * (-1) + 0.1 * 1 = -0.8$$

If the cat continues to choose white (because of the earlier episodes), it will get regrets of $1.6t$

Regrets

$$L_t = \sum_{i=1}^t (v_* - q(a_i)) = \sum_{a \in A} N_t(a) (v_* - q(a)) = \sum_{a \in A} N_t(a) \Delta_a$$

Now, minimizing the equation above shows the balance between exploration and exploitation.

If we want to take an action a many times, we need to reduce the action regret Δ_a .

Option 1: epsilon-greedy

A simple, yet powerful, algorithm

- $\epsilon \in [0,1]$ is a variable for exploration
- We select random action with probability ϵ
 - $a = \text{random}(\mathbf{A})$
- We select the best action with probability $1 - \epsilon$
 - $a = \text{argmax}_{a \in \mathbf{A}} Q_t(a)$

Option 2: Upper Confidence Bounds

Add an upper confidence term $U_t(a)$ to the action value and adjust it through learning

Then we select an action that minimizing the adjusted action value

$$a_t = \operatorname{argmax}_{a \in A} Q_t(a) + U_t(a)$$

Option 2: Upper Confidence Bounds

$$a_t = \operatorname{argmax}_{a \in A} Q_t(a) + U_t(a)$$

When we use action a a few times, $U_t(a)$ is large. (not sure if the value is correct)

When we use action a many times, $U_t(a)$ is small. (pretty sure that the value is correct)

Option 2: Upper Confidence Bounds

Assumption 1: we want to minimize this:

$$L_t = \sum_{i=1}^t (v_* - q(a_i)) = \sum_{a \in A} N_t(a) (v_* - q(a)) = \sum_{a \in A} N_t(a) \Delta_a$$

Assumption 2: we want to implement this:

When we use action a a few times, $U_t(a)$ is large. (not sure if the value is correct)

When we use action a many times, $U_t(a)$ is small. (pretty sure that the value is correct)

Option 2: Upper Confidence Bounds

$$U_t(a) = \sqrt{\frac{\log t}{2N_t(a)}}$$

Proved by using Hoeffding's Inequality

If we use the term above, we can implement an algorithm to both explore and exploit at the same time.

$$a_t = \operatorname{argmax}_{a \in A} Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}}$$

Option 2: Upper Confidence Bounds

$$a_t = \operatorname{argmax}_{a \in A} Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}}$$

Basically, when we haven't explored some actions much, the value is high, and that action is selected and updated. Eventually, the value will converge to the true value for each action. Thus we can find an optimal action.

Option 2: Upper Confidence Bounds

$$a_t = \operatorname{argmax}_{a \in A} Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}}$$

Basically, when we haven't explored some actions much, the value is high, and that action is selected and updated. Eventually, the value will converge to the true value for each action. Thus we can find an optimal action.